

MÓDULO 09: Introducción a Google Colaboratory (Colab) y Computación en la Nube

Curso: Python para Ciencia Abierta (CSIC)

Autor: Gustavo Liñán Cembrano

Fechas: 5 a 8 de Octubre de 2026

Tiempo Estimado de Impartición: 1 hora

Módulo: 09 - Cloud Computing, Google Colab, Kaggle API Integration, GPU/TPU Hardware Acceleration & Collaborative Data Science

1. Objetivos de Aprendizaje

- **Fundamentos de Computación en la Nube:** Comprender qué es **Google Colaboratory (Colab)**, su arquitectura sobre máquinas virtuales Linux en la nube y su papel democratizador en la investigación científica.
- **Aceleración por Hardware:** Conocer el uso de procesadores gráficos (GPU NVIDIA T4/L4) y unidades de procesamiento de tensores (TPU) de forma gratuita para cálculo masivo y Aprendizaje Profundo (*Deep Learning*).
- **Entorno Colaborativo e Inteligencia Artificial:** Trabajar de forma colaborativa simultánea (estilo Google Docs) e integrar asistentes de IA como **Gemini** para autocompletado y generación asistida de código.
- **Integración con Repositorios Externos y APIs:** Configurar la API de **Kaggle** mediante tokens de autenticación (`kaggle.json`) para descargar directamente datasets científicos sin ocupar espacio local.
- **Gestión de Almacenamiento y Persistencia:** Diferenciar entre el almacenamiento temporal de sesión (`/content/`), la vinculación con **Google Drive** (`drive.mount()`) y el trabajo en local con Antigravity-IDE / VS Code.
- **Análisis Exploratorio de Datos (EDA) en la Nube:** Cargar, traducir, limpiar y visualizar conjuntos de datos biomédicos reales (*Thyroid Disease Dataset*) mediante Pandas, Seaborn y Matplotlib.

⚠ REQUISITO FUNDAMENTAL PARA EL ALUMNADO:

Para poder ejecutar y aprovechar al máximo las funcionalidades de este módulo (aceleración por GPU/TPU, integración con Gemini AI y ejecución de comandos Linux con `!`), **este cuaderno debe subirse y ejecutarse directamente en la plataforma Google Colab.**

Instrucciones Paso a Paso para Subir y Abrir este Cuaderno en Google Colab:

1. Accede a Google Colab en tu Navegador:

- Abre tu navegador web (Chrome, Firefox o Edge) e ingresa en:
<https://colab.research.google.com/>.
- Inicia sesión con tu cuenta personal de Google o tu cuenta institucional de Google Workspace (ej. CSIC).

2. Subir el Cuaderno desde tu Equipo:

- En la ventana emergente que aparece al acceder, selecciona la pestaña "**Subir**" (*Upload*).
- Haz clic en "**Seleccionar archivo**" (o arrastra el archivo directamente a la ventana).
- Busca y selecciona el archivo **09_INTRO_GOOGLE_COLAB.ipynb** situado dentro de la carpeta **NOTEBOOKS/** de tu ordenador.

3. Configurar el Entorno de Ejecución (Opcional - Aceleración por GPU/TPU):

- Una vez abierto el cuaderno en Colab, dirígete al menú superior:
Entorno de ejecución → **Cambiar tipo de entorno de ejecución** (*Runtime* → *Change runtime type*).
- En el desplegable **Acelerador de hardware**, puedes seleccionar **GPU (T4)** para tareas computacionales pesadas o mantener **CPU** para análisis estándar.

4. Ejecutar Celdas y Trabajar en la Nube:

- ¡Ya estás conectado a una máquina virtual de Google!
- Puedes ejecutar cada celda de código seleccionándola y pulsando **Shift + Enter** o haciendo clic en el botón de reproducción  a la izquierda de la celda.

2. ¿Qué es Google Colab? Ventajas, Inconvenientes y Comparativa

Google Colaboratory (Colab) es un servicio de cuaderno de Jupyter alojado en la nube por Google. Permite escribir, ejecutar y compartir código Python directamente en el navegador web sin necesidad de instalar o configurar ningún software localmente.

Cada cuaderno de Colab se ejecuta en una **máquina virtual Linux dedicada** (con recursos de CPU, RAM y espacio en disco) provista dinámicamente por los servidores de Google.

Ventajas Principales para la Ciencia Abierta:

1. **Configuración Cero:** No requiere instalar Python, entornos virtuales ni librerías locales. Con solo una cuenta de Google puedes empezar a codificar.
2. **Acceso Gratuito a Hardware de Alto Rendimiento (GPU/TPU):** Proporciona acceso a aceleradores de hardware (GPU NVIDIA T4/L4 o TPU) esenciales para *Machine Learning* y simulaciones pesadas.
3. **Ecosistema Científico Preinstalado:** Incluye por defecto las principales librerías de datos (`pandas` , `numpy` , `scipy` , `scikit-learn` , `matplotlib` , `seaborn` , `tensorflow` , `torch`).
4. **Colaboración en Tiempo Real:** Permite que varios investigadores trabajen, comenten y editen el mismo cuaderno simultáneamente, con control de avatares visibles en la interfaz.
5. **Integración con Asistentes de IA (Gemini):** Incorpora ayuda en línea con Gemini para autocompletar código, explicar errores y sugerir refactorizaciones.
6. **Persistencia e Integración:** Guardado automático en **Google Drive** y exportación directa a repositorios de **GitHub**.

Inconvenientes y Limitaciones a Tener en Cuenta:

1. **Sesiones Efímeras (Tiempo de Ejecución):** Las instancias en la nube son temporales. Si el cuaderno permanece inactivo o supera el tiempo máximo por sesión (generalmente 12 horas en la versión gratuita), la máquina se desconecta y los archivos locales del directorio `/content/` se borran.
2. **Dependencia de Conexión a Internet:** Requiere conexión continua y estable a la red.
3. **Políticas de Privacidad y Confidencialidad:** Al subir datos a Colab, estos residen temporalmente en la infraestructura de Google. Se debe verificar el nivel de sensibilidad y protección de datos antes de subir proyectos confidenciales del CSIC.

Comparativa: Google Colab vs. Entorno Local (Antigravity-IDE / VS Code)

Característica	Google Colab (Nube)	Entorno Local (Antigravity-IDE / VS Code)
Instalación y Setup	 Ninguna (100% en el navegador).	 Requiere instalar Python, <code>venv</code> , extensiones e IDE.
Aceleración Hardware	 GPU/TPU gratuitas incluidas.	 Limitado a la potencia del equipo físico local.
Persistencia de Datos	 Almacenamiento de sesión efímero (<code>/content/</code>).	 Almacenamiento permanente en el disco duro local.

Característica	Google Colab (Nube)	Entorno Local (Antigravity-IDE / VS Code)
Colaboración	 Simultánea en tiempo real.	 Asíncrona mediante Git / GitHub.
Privacidad de Datos	 Aloja datos en servidores de Google.	 Máxima privacidad (los datos no salen del equipo/CSIC).
Uso sin Internet	 Imposible (Requiere conexión).	 100% Funcional sin conexión a internet.

3. Integración con Repositorios Externos mediante la API de Kaggle

En muchos proyectos científicos deseamos descargar conjuntos de datos directamente desde plataformas públicas como **Kaggle** sin necesidad de descargarlos primero a nuestro ordenador local.

Conceptos Clave:

- **API (Application Programming Interface):** Conjunto de reglas y protocolos que permite a dos programas informáticos comunicarse entre sí programáticamente.
- **Clave/Token API (`kaggle.json`):** Archivo de credenciales secretas en formato JSON que autentica a tu usuario ante los servidores de Kaggle de forma segura.



Pasos para Autenticar Kaggle en Colab:

1. Entra en [Kaggle.com](https://kaggle.com) e inicia sesión.
2. Ve a **Account / Settings** → Sección **API** → Haz clic en **Create New API Token**.
Esto descargará el archivo `kaggle.json`.
3. En el panel izquierdo de Colab, abre el icono de la **carpeta** (Explorador de archivos) y sube `kaggle.json`.
4. Ejecuta las siguientes celdas para mover las credenciales al directorio del sistema `~/.kaggle/` y ajustar sus permisos de seguridad (`chmod 600`).

```
In [ ]: # 1. Instalación de la librería oficial de Kaggle mediante comando de consola
!pip install --quiet kaggle
print("Librería de Kaggle instalada exitosamente.")
```

```
In [ ]: # 2. Configuración segura del token API en el sistema Linux de Colab
# NOTA: Esta celda moverá kaggle.json si ha sido subido a /content/
import os

if os.path.exists('/content/kaggle.json') or os.path.exists('kaggle.json'):
    !mkdir -p ~/.kaggle/
    !cp /content/kaggle.json ~/.kaggle/ 2>/dev/null || cp kaggle.json ~/.kaggle/
```

```

!chmod 600 ~/.kaggle/kaggle.json
print("Token API de Kaggle configurado correctamente en ~/.kaggle/kaggle
else:
print("💡 Nota pedagógica: Para descargar desde la API real de Kaggle en

```

```

In [ ]: # 3. Descarga y extracción programática del dataset biomédico desde Kaggle
# Comando: !kaggle datasets download -d jainaru/thyroid-disease-data

dataset_zip = "thyroid-disease-data.zip"
csv_file = "Thyroid_Diff.csv"

# Verificamos si ya tenemos el CSV localmente (en DATASETS/ o directorio act
datasets_dir = "./DATASETS"
local_csv_path = os.path.join(datasets_dir, csv_file)

if os.path.exists(local_csv_path):
    print(f"Dataset local encontrado en '{local_csv_path}'. No es necesario
else:
    # Intento de descarga programática vía Kaggle si el token existe
    if os.path.exists(os.path.expanduser('~/.kaggle/kaggle.json')):
        !kaggle datasets download -d jainaru/thyroid-disease-data
        !unzip -o thyroid-disease-data.zip
        print("Dataset descargado y descomprimido exitosamente desde Kaggle.
    else:
        print("Utilizando copia de seguridad disponible en el entorno.")

```

```

In [ ]: !mkdir DATASETS
!mv Thyroid_Diff.csv DATASETS/

target_csv = os.path.join(datasets_dir, csv_file)

```

4. Análisis Exploratorio de Datos (EDA) en la Nube: Cáncer de Tiroides

Analizaremos un conjunto de datos clínico real obtenido de Kaggle/UCI sobre la **recurrencia del cáncer de tiroides bien diferenciado**. Contiene 13 variables clínico-patológicas recopiladas durante 15 años con un seguimiento mínimo de 10 años por paciente.

Atributos Clave:

- **Age / Gender / Smoking** : Edad, género e historial de tabaquismo.
- **Thyroid Function** : Evaluación clínica de la función tiroidea.
- **Physical Examination** : Diagnóstico físico de bocio o nódulos.
- **Adenopathy** : Presencia o ausencia de adenopatías linfáticas en el cuello.
- **Pathology** : Histología tumoral (Papilar, Folicular, Células de Hurthle, Micro-papilar).
- **Risk** : Clasificación de riesgo de recurrencia (Bajo, Intermedio, Alto).

- **T, N, M** : Sistema de clasificación TNM de tumores sólidos.
- **Response** : Respuesta al tratamiento (Excelente, Indeterminada, Incompleta).
- **Reccured** : Recurrencia de la enfermedad (Sí / No).

```
In [ ]: # Importación de librerías para análisis y visualización
import sys
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Configuración de estilo gráfico limpio
sns.set_theme(style="whitegrid", palette="muted")
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'

df = pd.read_csv(target_csv)

print(f"Forma del DataFrame: {df.shape[0]} filas, {df.shape[1]} columnas")
print("Primeras 5 filas del dataset original:")
print(df.head(5))
```

```
In [ ]: # Diagnóstico de valores nulos o ausentes
nas_totales = df.isna().sum().sum()
nulls_totales = df.isnull().sum().sum()

print("=" * 60)
print(f"Recuento total de valores NA: {nas_totales}")
print(f"Recuento total de valores NULL: {nulls_totales}")

if nas_totales == 0 and nulls_totales == 0:
    print("✅ El conjunto de datos está 100% completo (sin valores faltantes)")
else:
    print("⚠️ Se requieren tareas de imputación o limpieza de nulos.")
print("=" * 60)
```

5. Curación de Metadatos: Traducción de Columnas y Categorías

Para facilitar la interpretación científica del equipo clínico en España, traduciremos y normalizaremos los nombres de las columnas y los valores de las variables categóricas al español.

```
In [ ]: # Renombrar columnas a términos científicos en español
columnas_es = {
    'Age': 'Edad',
    'Gender': 'Género',
    'Smoking': 'Fumador',
    'Hx Smoking': 'H. tabaquismo',
    'Hx Radiothreapy': 'H. radioterapia',
    'Thyroid Function': 'Función tiroidea',
```

```

    'Physical Examination': 'Examen físico',
    'Adenopathy': 'Adenopatía',
    'Pathology': 'Patología',
    'Focality': 'Focalidad',
    'Risk': 'Riesgo',
    'T': 'Tumor',
    'N': 'Ganglio Linfático',
    'M': 'Metastasis',
    'Stage': 'Estadio',
    'Response': 'Respuesta',
    'Recurrent': 'Recurrente'
}

```

```

df.rename(columns=columnas_es, inplace=True)
print("Columnas renombradas exitosamente:")
print(list(df.columns))

```

```

In [ ]: # Normalización y traducción de valores categóricos clave
df['Adenopatía'] = df['Adenopatía'].replace({
    'No': 'Sin Adenopatía',
    'Left': 'Lado Izquierdo',
    'Right': 'Lado Derecho',
    'Extensive': 'Extensa y Generalizada'
})

df['Función tiroidea'] = df['Función tiroidea'].replace({
    'Normal': 'Normal',
    'Euthyroid': 'Eutiroidismo',
    'Clinical Hyperthyroidism': 'Hipertiroidismo Clínico',
    'Clinical Hypothyroidism': 'Hipotiroidismo Clínico',
    'Subclinical Hyperthyroidism': 'Hipertiroidismo Subclínico',
    'Subclinical Hypothyroidism': 'Hipotiroidismo Subclínico'
})

df['Examen físico'] = df['Examen físico'].replace({
    'Single nodular goiter-left': 'Bocio nodular único - izq.',
    'Multinodular goiter': 'Bocio multinodular',
    'Single nodular goiter-right': 'Bocio nodular único - der.',
    'Diffuse goiter': 'Bocio difuso',
    'Normal': 'Normal'
})

df['Patología'] = df['Patología'].replace({
    'Micropapillary': 'Micro-papilar',
    'Papillary': 'Papilar',
    'Follicular': 'Folicular',
    'Hurthel cell': 'Células de Hurthle'
})

df['Riesgo'] = df['Riesgo'].replace({
    'Low': 'Bajo',
    'Intermediate': 'Intermedio',
    'High': 'Alto'
})

df['Respuesta'] = df['Respuesta'].replace({

```

```

    'Indeterminate': 'Indeterminada',
    'Excellent': 'Excelente',
    'Structural Incomplete': 'Incompleta Estructural',
    'Biochemical Incomplete': 'Incompleta Bioquímica'
})

# Estandarizar respuestas binarias Yes/No a s/n
df.replace({'Yes': 's', 'No': 'n'}, inplace=True)

print("Categorías traducidas correctamente. Muestra de 'Respuesta':")
print(df['Respuesta'].value_counts())

```

```

In [ ]: # Inserción de un nuevo registro de ensayo clínico (Simulación)
paciente_control = {
    'Edad': 47,
    'Género': 'M',
    'Fumador': 'n',
    'H. tabaquismo': 'n',
    'H. radioterapia': 'n',
    'Función tiroidea': 'Eutiroidismo',
    'Examen físico': 'Bocio nodular único - der.',
    'Adenopatía': 'Sin Adenopatía',
    'Patología': 'Papilar',
    'Focalidad': 'Uni-Focal',
    'Riesgo': 'Bajo',
    'Tumor': 'T2a',
    'Ganglio Linfático': 'N0',
    'Metastasis': 'M0',
    'Estadio': 'II',
    'Respuesta': 'Excelente',
    'Recurrente': 'n'
}

df = pd.concat([df, pd.DataFrame([paciente_control])], ignore_index=True)

# Exportar dataset curado
output_curated_path = os.path.join(datasets_dir, "Thyroid_Diff_ES.csv")
df.to_csv(output_curated_path, index=False)

print(f"Nuevo registro concatenado. Dataset guardado en: '{output_curated_pa

```

6. Visualización de Patrones Clínicos: Seaborn y Matplotlib en Colab

Generaremos figuras compuestas para analizar la relación entre la edad de los pacientes, el estadio del tumor, la categoría de riesgo y la respuesta al tratamiento.

```

In [ ]: # 1. Diagramas de Caja (Boxplots): Distribución de Edad por Estadio del Tumor
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Boxplot por Tumor
sns.boxplot(x='Tumor', y='Edad', data=df, palette="OrRd", hue='Tumor', legend=
axes[0].set_title('Distribución de Edad por Clasificación de Tumor (T)', for

```

```

axes[0].set_xlabel('Estadio del Tumor')
axes[0].set_ylabel('Edad del Paciente')
axes[0].tick_params(axis='x', rotation=45)

# Boxplot por Estadio General
sns.boxplot(x='Estadio', y='Edad', data=df, palette="PuBu", hue='Estadio', 1
axes[1].set_title('Distribución de Edad por Estadio Clínico General', fontwe
axes[1].set_xlabel('Estadio General')
axes[1].set_ylabel('Edad del Paciente')

plt.tight_layout()
plt.show()

```

```

In [ ]: # 2. Histogramas de Distribución por Edad y Género
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Histograma de Edad con Estimación de Densidad de Kernel (KDE)
sns.histplot(df['Edad'], bins=20, kde=True, color='steelblue', ax=axes[0])
axes[0].set_title('Distribución Etaria de la Cohorte', fontweight='bold')
axes[0].set_xlabel('Edad')
axes[0].set_ylabel('Frecuencia (Nº Pacientes)')

# Conteo por Género
sns.countplot(x='Género', data=df, palette=['pink', 'lightblue'], ax=axes[1])
axes[1].set_title('Distribución por Género', fontweight='bold')
axes[1].set_xlabel('Género (F: Femenino, M: Masculino)')
axes[1].set_ylabel('Frecuencia')

plt.tight_layout()
plt.show()

```

```

In [ ]: # 3. Gráficos de Violín (Violin Plots): Edad por Categoría de Riesgo
fig, ax = plt.subplots(figsize=(9, 5))

sns.violinplot(x='Riesgo', y='Edad', data=df, inner='quartile', palette='Set
ax.set_title('Distribución de Edades por Categoría de Riesgo Clínico', fonts
ax.set_xlabel('Categoría de Riesgo')
ax.set_ylabel('Edad')

plt.show()

```

7. Ejercicios Prácticos y Autoevaluación

Ejercicio 1: Cálculo de Porcentajes de Recurrencia

Calcula el porcentaje de pacientes que experimentaron recurrencia (`Recurrente == 's'`) frente a los que no (`Recurrente == 'n'`) y represéntalo mediante un gráfico de barras.

```

In [ ]: # Solución Ejercicio 1
recurrencia_pct = df['Recurrente'].value_counts(normalize=True) * 100

```

```
fig, ax = plt.subplots(figsize=(6, 4))
barras = ax.bar(['Sin Recurrencia (n)', 'Con Recurrencia (s)'], recurrencia_)

ax.set_title('Porcentaje de Recurrencia del Cáncer de Tiroides', fontweight=
ax.set_ylabel('Porcentaje (%)')
ax.set_ylim(0, 100)
ax.bar_label(barras, fmt='%.1f%%', padding=4, fontweight='bold')

plt.tight_layout()
plt.show()
```



ANEXO INTERACTIVO (15 MIN): Widgets de `ipywidgets` y Formularios Dinámicos en Google Colab

Curso: Python para Ciencia Abierta (CSIC)

Autor: Gustavo Liñán Cembrano

Fechas: 5 a 8 de Octubre de 2026

Tiempo Estimado de Impartición: 15 minutos (Anexo Módulo 09)

Temática: Interactive Widgets (`ipywidgets`), Google Colab Forms (`#@param`), Dynamic Data Filtering & Real-time Plot Dashboards

1. Objetivos de Aprendizaje

- **Formularios Nativos de Google Colab (`#@param`):** Convertir variables de código en controles gráficos dinámicos (deslizadores, despleables, casillas de verificación y campos de texto) sin programar interfaz web.
- **Controles Interactivos Estándar (`ipywidgets`):** Utilizar la librería universal `ipywidgets` y el decorador `@interact` para crear paneles interactivos compatibles con Colab, Jupyter y VS Code.
- **Dashboards de Exploración Clínica:** Vincular controles deslizantes y listas despleables con algoritmos de filtrado en Pandas y gráficos en Seaborn/Matplotlib que se actualizan en tiempo real.
- **Ejecución Bajo Demanda (`interact_manual`):** Controlar la renderización de análisis pesados mediante botones de ejecución dinámica.

2. Formularios Nativos de Google Colab (`#@param`)

En Google Colab, añadir comentarios especiales `#@param` tras una variable transforma automáticamente el código Python en un **formulario gráfico** en la parte derecha de la

celda.

Tipos de Campos Comunes:

- **Deslizador numérico:** `edad = 35 #@param {type:"slider", min:18, max:90, step:1}`
- **Menú desplegable:** `riesgo = "Bajo" #@param ["Bajo", "Intermedio", "Alto"]`
- **Casilla de verificación:** `fumador = True #@param {type:"boolean"}`
- **Texto:** `paciente_id = "PAC-001" #@param {type:"string"}`

```
In [ ]: # @title 📄 Formulario Clínico de Filtrado Rápido (Colab Form)
# @markdown Configura los parámetros en el panel de la derecha para filtrar

edad_maxima = 65 # @param {type:"slider", min:18, max:90, step:1}
categoria_riesgo = "Bajo" # @param ["Todos", "Bajo", "Intermedio", "Alto"]
incluir_fumadores = True # @param {type:"boolean"}
patologia_select = "Papilar" # @param ["Todas", "Papilar", "Folicular", "Mi

print(f"🔍 Criterios seleccionados:")
print(f" - Edad Máxima: {edad_maxima} años")
print(f" - Categoría de Riesgo: {categoria_riesgo}")
print(f" - Incluir Fumadores: {incluir_fumadores}")
print(f" - Patología: {patologia_select}")
```

3. Dashboards Dinámicos con `ipywidgets` y `@interact`

`ipywidgets` es el estándar abierto de la comunidad Jupyter. El decorador `@interact` inspecciona los argumentos de una función de Python y crea automáticamente los controles adecuados.

```
In [ ]: # Carga e Ingesta del Dataset Curado (Thyroid_Diff_ES.csv)
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from ipywidgets import interact, IntSlider, Dropdown, ToggleButtons

# Estilo gráfico limpio
sns.set_theme(style="whitegrid", palette="muted")

# Determinar ruta flexible del dataset
datasets_dir = "../DATASETS" if os.path.exists("../DATASETS") else "DATASETS"
csv_path = os.path.join(datasets_dir, "Thyroid_Diff_ES.csv")

if not os.path.exists(csv_path):
    csv_path = os.path.join(datasets_dir, "Thyroid_Diff.csv")
```

```
df_widget = pd.read_csv(csv_path if os.path.exists(csv_path) else "Thyroid_D
print(f"Dataset cargado para exploración interactiva. Registros totales: {le
```

```
In [ ]: # Dashboard Clínico Interactivo en Tiempo Real
@interact(
    edad_limite=IntSlider(min=25, max=80, step=5, value=55, description='Eda
    filtro_riesgo=Dropdown(options=['Todos', 'Bajo', 'Intermedio', 'Alto'],
    vista_grafico=ToggleButtons(options=['Boxplot', 'Violinplot', 'Histogram
)
def dashboard_clinico_interactivo(edad_limite, filtro_riesgo, vista_grafico)
    # 1. Filtrar DataFrame dinámicamente
    df_sub = df_widget[df_widget['Edad'] <= edad_limite].copy()
    if filtro_riesgo != 'Todos' and 'Riesgo' in df_sub.columns:
        df_sub = df_sub[df_sub['Riesgo'] == filtro_riesgo]

    print(f"📊 Muestra filtrada: {len(df_sub)} pacientes (Edad <= {edad_limite})

    # 2. Renderizar gráfico dinámico
    fig, ax = plt.subplots(figsize=(10, 4.5))

    if vista_grafico == 'Boxplot':
        col_x = 'Estadio' if 'Estadio' in df_sub.columns else 'Stage'
        sns.boxplot(x=col_x, y='Edad', data=df_sub, palette='Set2', hue=col_x)
        ax.set_title(f'Distribución de Edad por Estadio Clínico (Riesgo: {filtro_riesgo})')
    elif vista_grafico == 'Violinplot':
        col_x = 'Respuesta' if 'Respuesta' in df_sub.columns else 'Response'
        sns.violinplot(x=col_x, y='Edad', data=df_sub, palette='Pastel1', ax=ax)
        ax.set_title(f'Edad por Respuesta al Tratamiento (Riesgo: {filtro_riesgo})')
        ax.tick_params(axis='x', rotation=20)
    else:
        sns.histplot(df_sub['Edad'], bins=15, kde=True, color='teal', ax=ax)
        ax.set_title(f'Histograma (N = {len(df_sub)})', fontweight='bold')
        ax.set_xlabel('Edad (años)')
        ax.set_ylabel('Nº de Pacientes')

    plt.tight_layout()
    plt.show()
```

4. Visualizador Multivariable con Botón de Ejecución (interact_manual)

Cuando el análisis o la generación de figuras es computacionalmente intensivo, es recomendable utilizar `interact_manual`. Esto añade un botón **"Run Interact"** para que el gráfico solo se vuelva a calcular cuando el usuario confirme los cambios.

```
In [ ]: # Generador de Figuras Bajo Demanda (interact_manual)
from ipywidgets import interact_manual

@interact_manual(
    variable_x=Dropdown(options=['Estadio', 'Tumor', 'Riesgo', 'Patología'],
```

```

    paleta_color=Dropdown(options=['viridis', 'magma', 'Set2', 'PuBu', 'Spec
    mostrar_puntos=True
)
def reporte_multivariable_manual(variable_x, paleta_color, mostrar_puntos):
    if variable_x not in df_widget.columns:
        print(f"La columna '{variable_x}' no se encuentra en el DataFrame.")
        return

    fig, ax = plt.subplots(figsize=(10, 5))

    # Boxplot base
    sns.boxplot(x=variable_x, y='Edad', data=df_widget, palette=paleta_color)

    # Superponer puntos individuales (Stripplot) si está marcado
    if mostrar_puntos:
        sns.stripplot(x=variable_x, y='Edad', data=df_widget, color='black',

    ax.set_title(f'Análisis Exploratorio: Edad vs. {variable_x}', fontsize=1
    ax.set_xlabel(variable_x)
    ax.set_ylabel('Edad del Paciente')
    ax.tick_params(axis='x', rotation=30)

    plt.tight_layout()
    plt.show()

```

```

In [ ]: !rm -Rf ./DATASETS/
        !rm thyroid-disease-data.zip

```